



Chp07 - these are answers to the review questions

Database Design and Implementation (University of South Africa)



Scan to open on Studocu

Chapter 7

Introduction to Structured Query Language (SQL)

NOTE

Several points are worth emphasizing:

- We have provided the SQL scripts for both chapters 7 and 8. These scripts are intended to facilitate the flow of the material presented to the class. However, given the comments made by our students, the scripts should **not** replace the manual typing of the SQL commands by students. Some students learn SQL better when they have a chance to type their own commands and get the feedback provided by their errors. We recommend that the students use their lab time to practice the commands manually.
- Because this chapter focuses on learning SQL, we recommend that you use the Microsoft Access SQL window to type SQL queries. Using this approach, you will be able to demonstrate the interoperability of standard SQL. For example, you can cut and paste the same SQL command from the SQL query window in Microsoft Access, to Oracle SQL * Plus and to MS SQL Query Analyzer. This approach achieves two objectives:
 - It demonstrates that adhering to the SQL standard means that most of the SQL code will be portable among DBMSes.
 - It also demonstrates that even a widely accepted SQL standard is sometimes implemented with slight distinctions by different vendors. For example, the treatment of date formats in Microsoft Access and Oracle is slightly different.

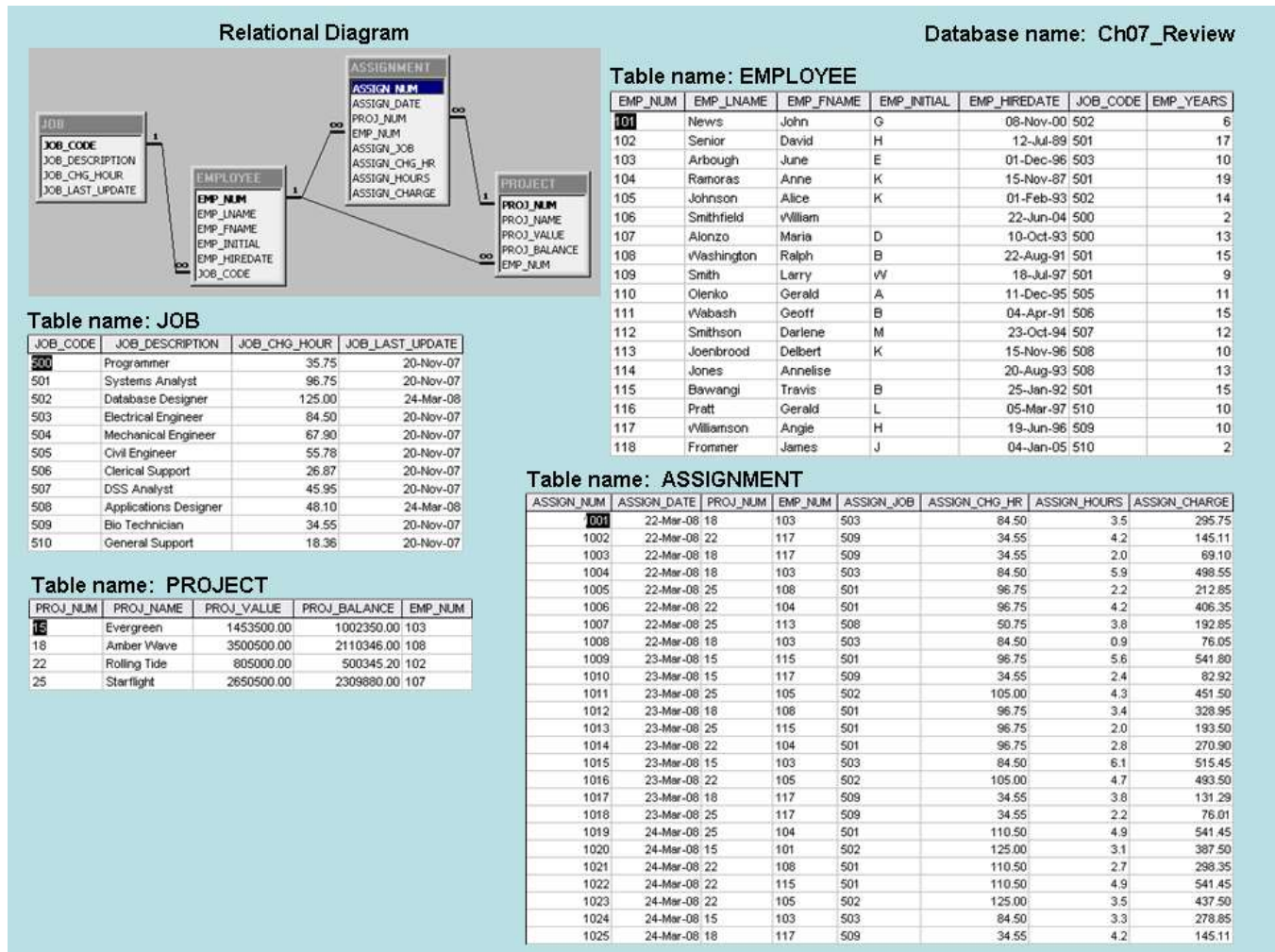
Answers to Review Questions



ONLINE CONTENT

The Review Questions in this chapter are based on the [Ch07_Review](#) database located in the Online Student Companion. This database is stored in Microsoft Access format. The Student Online Companion also includes SQL script files (Oracle and SQLServer) for all of the data sets used throughout the book.

The [Ch07_Review](#) database stores data for a consulting company that tracks all charges to projects. The charges are based on the hours each employee works on each project. The structure and contents of the [Ch07_Review](#) database are shown in Figure Q7.1.

FIGURE Q7.1 Structure and Contents of the Ch07_Review Database

As you examine Figure Q7.1, note that the ASSIGNMENT table stores the JOB charge per hour the employee job code values as a “native” attribute to maintain historical accuracy of the data. The JOB_CHG_HOUR values are likely to change over time. In fact, we have made sure that there is a JOB_CHG_HOUR change that will be reflected in the ASSIGNMENT table. And, naturally, the employee primary job assignment may change, so the ASSIGN_JOB is also stored. Because these attributes are required to maintain the historical accuracy of the data, they are *not* redundant. For example, note that the JOB table was updated on 24-Mar-2008 for JOB_CODE 502 (Database Designer) and shows a JOB_CHG_HOUR of \$125.00. The ASSIGNMENT table shows this new charge in its ASSIGN_CHG_HR, starting on 24-Mar-2007. (Look at records 1020 and 1023 in the ASSIGNMENT table. The “old” ASSIGN_CHG_HR was \$105.00 per hour – see records 1011 and 1016 for 23-Mar-2008.)

Given the structure and contents of the Ch07_Review database shown in Figure Q7.1, use SQL commands to answer questions 1–25.

1. Write the SQL code that will create the table structure for a table named **EMP_1**. This table is a subset of the **EMPLOYEE** table. The basic **EMP_1** table structure is summarized in Table Q7.1. (Note that the **JOB_CODE** is the FK to **JOB**.)

Table Q7.1 The EMP_1 Table Structure

ATTRIBUTE (FIELD) NAME	DATA DECLARATION
EMP_NUM	CHAR(3)
EMP_LNAME	VARCHAR(15)
EMP_FNAME	VARCHAR(15)
EMP_INITIAL	CHAR(1)
EMP_HIREDATE	DATE
JOB_CODE	CHAR(3)

```
CREATE TABLE EMP_1 (
EMP_NUM          CHAR(3)          PRIMARY KEY,
EMP_LNAME        VARCHAR(15)      NOT NULL,
EMP_FNAME        VARCHAR(15)      NOT NULL,
EMP_INITIAL      CHAR(1),
EMP_HIREDATE     DATE,
JOB_CODE         CHAR(3),
FOREIGN KEY (JOB_CODE) REFERENCES JOB);
```

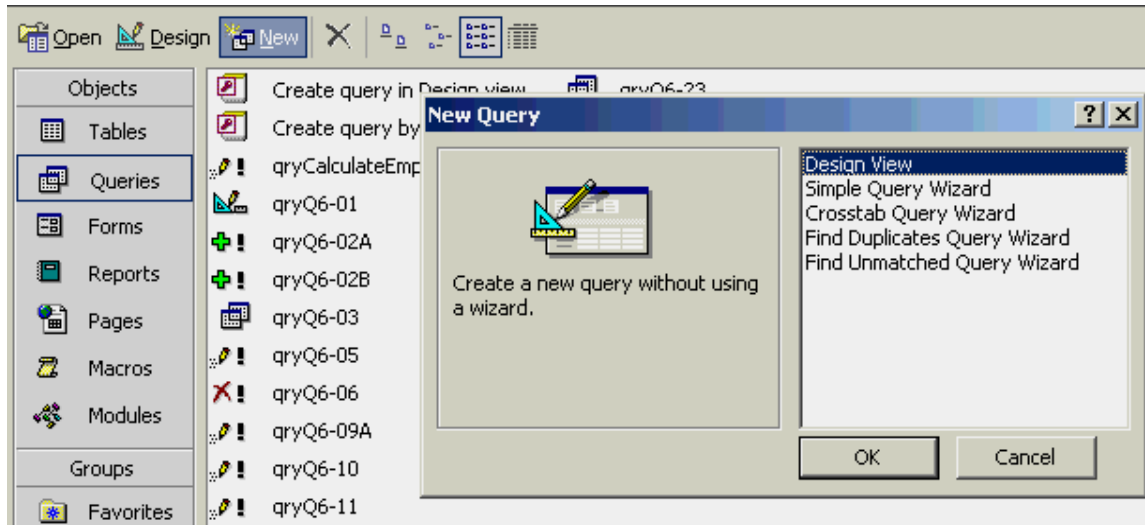
NOTE

We have already provided the **EMP_1** table for you. If you try to run the preceding query, you will get an error message because the **EMP_1** table already exists. If you want to go ahead and run the SQL code we have created for you in the **Ch07_Review** database – the query name is **qryQ6-01** --change the name of the table in the **CREATE TABLE** command to **EMP_TEST**. After executing the query, you can check the **EMP_TEST** table structure. The SQL code only created the **EMP_TEST** table *structure*, so it will not have any data in it.

The preceding SQL command lines were created by copying the SQL code from the **qryQ6-01** query. If you use MS Access to let your students generate the SQL code, follow the sequence shown in Figures Q7.1S1 through Q7.1S5.

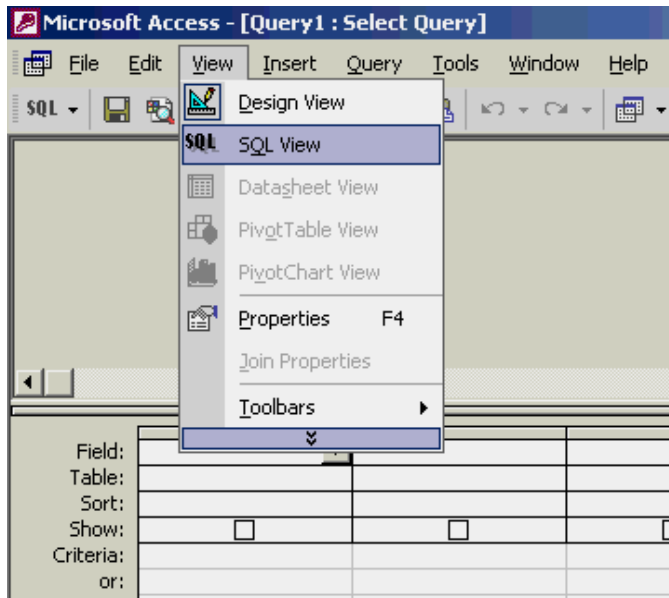
Step 1: Select the **Queries/New/Design** option shown in Figure Q7.1S1.

Figure Q7.1S1 Preparing to Write the SQL Code



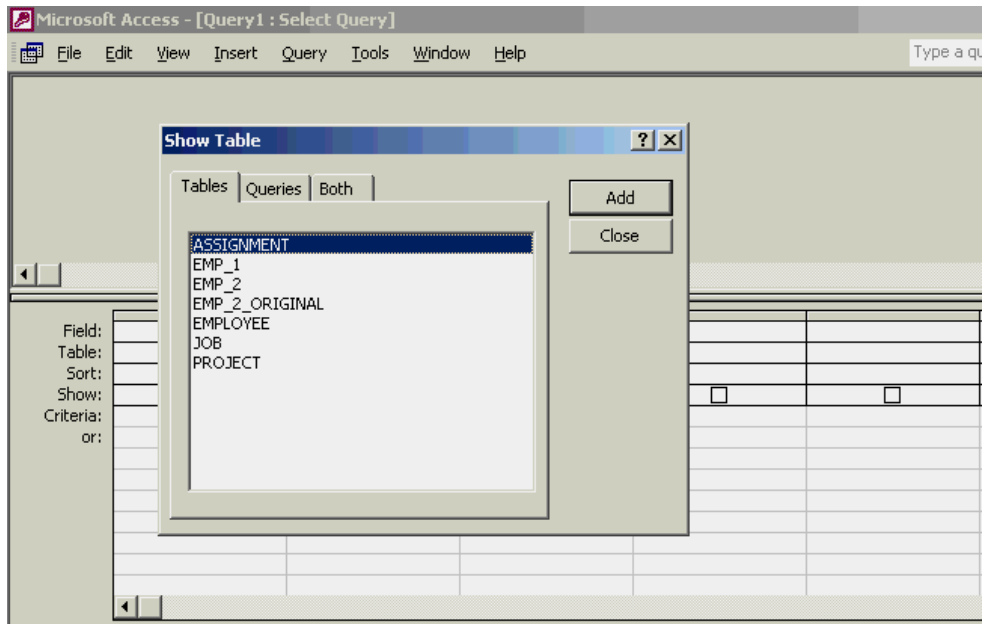
Step 2: Select the **View/SQL View** option shown in Figure Q7.1S2. Selecting this option opens the Query by Example (QBE) screen shown in Figure Q7.1S3.

Figure Q7.1S2 Select the SQL Option



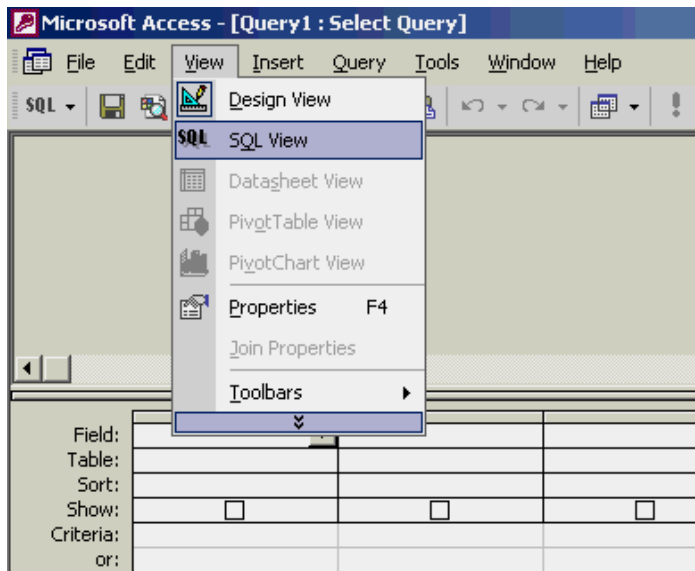
Step 3: Click on the **Close** button shown in Figure Q7.1S3 to close the **Show Table** window. (Note that the **Show Table** window shows all of the available tables. If you select the **Queries** tab, you will see all available queries. Remember, the objective is to let your students write standard SQL code, so *you don't want them to use the QBE option to let MS Access generate the MS Access version of SQL.*) This selection generates the screen shown in Figure Q7.1S4.

Figure Q7.1S3 The QBE Screen



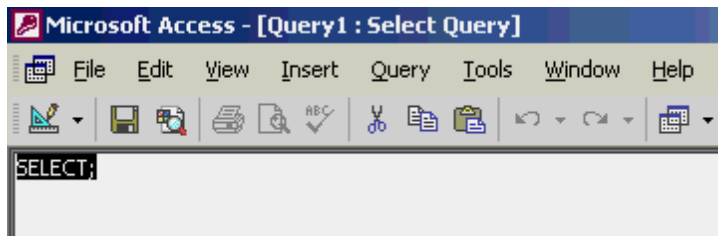
Step 4: Select the [View/SQL View](#) sequence shown in Figure Q7.1S4 to generate the screen shown in Figure Q7.S5.

Figure Q7.1S4 Select the SQL View



Step 5: Start typing your SQL code.

Figure Q7.1S5 Ready to Type SQL Code



When the SQL code has been typed, save the query. MS Access will prompt you if you forget to save your efforts. If you have made a syntax error (or some other error) in the SQL code, MS Access will indicate the likely nature of the error.

2. Having created the table structure in Question 1, write the SQL code to enter the first two rows for the table shown in Figure Q7.2.

FIGURE Q7.2 The Contents of the EMP_1 Table

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_HIREDATE	JOB_CODE
101	News	John	G	08-Nov-00	502
102	Senior	David	H	12-Jul-89	501
103	Arbough	June	E	01-Dec-96	500
104	Ramoras	Anne	K	15-Nov-87	501
105	Johnson	Alice	K	01-Feb-93	502
106	Smithfield	William		22-Jun-04	500
107	Alonzo	Maria	D	10-Oct-93	500
108	Washington	Ralph	B	22-Aug-91	501
109	Smith	Larry	W	18-Jul-97	501

```
INSERT INTO EMP_1 VALUES ('101', 'News', 'John', 'G', '08-Nov-00', '502');
INSERT INTO EMP_1 VALUES ('102', 'Senior', 'David', 'H', '12-Jul-89', '501');
```

3. Assuming that the data shown in the EMP_1 table have been entered, write the SQL code that will list all attributes for a job code of 502.

```
SELECT *
FROM EMP_1
WHERE JOB_CODE = '502';
```

4. Write the SQL code that will save the changes made to the EMP_1 table.

```
COMMIT;
```

5. Write the SQL code to change the job code to 501 for the person whose personnel number is 107. After you have completed the task, examine the results, and then reset the job code to its original value.

```
UPDATE EMP_1
SET      JOB_CODE = '501'
WHERE    EMP_NUM = '107';
```

To see the changes:

```
SELECT  *
FROM    EMP_1
WHERE   EMP_NUM = '107';
```

To reset, use

```
ROLLBACK;
```

6. Write the SQL code to delete the row for the person named William Smithfield, who was hired on June 22, 2004 and whose job code classification is 500. (*Hint: Use logical operators to include all the information given in this problem.*)

```
DELETE FROM EMP_1
WHERE   EMP_LNAME = 'Smithfield'
AND     EMP_FNAME = 'William'
AND     EMP_HIREDATE = '22-June-04'
AND     JOB_CODE = '500';
```

7. Write the SQL code that will restore the data to its original status; that is, the table should contain the data that existed before you made the changes in Questions 5 and 6.

```
ROLLBACK;
```

8. Write the SQL code to create a copy of EMP_1, naming the copy EMP_2. Then write the SQL code that will add the attributes EMP_PCT and PROJ_NUM to its structure. The EMP_PCT is the bonus percentage to be paid to each employee. The new attribute characteristics are:

```
EMP_PCT      NUMBER(4,2)
PROJ_NUM     CHAR(3)
```

(*Note: If your SQL implementation allows it, you may use DECIMAL(4,2) rather than NUMBER(4,2).*)

There are two way to get this job done. The two possible solutions are shown next.

Solution A:

```
CREATE TABLE EMP_2 (  
    EMP_NUM          CHAR(3)          NOT NULL UNIQUE,  
    EMP_LNAME        VARCHAR(15)      NOT NULL,  
    EMP_FNAME        VARCHAR(15)      NOT NULL,  
    EMP_INITIAL       CHAR(1),  
    EMP_HIREDATE      DATE             NOT NULL,  
    JOB_CODE          CHAR(3)          NOT NULL,  
    PRIMARY KEY (EMP_NUM),  
    FOREIGN KEY (JOB_CODE) REFERENCES JOB);
```

```
INSERT INTO EMP_2 SELECT * FROM EMP_1;
```

```
ALTER TABLE EMP_2  
    ADD (EMP_PCT      NUMBER (4,2)),  
    ADD (PROJ_NUM     CHAR(3));
```

Solution B:

```
CREATE TABLE EMP_2 AS SELECT * FROM EMP_1;
```

```
ALTER TABLE EMP_2  
    ADD (EMP_PCT      NUMBER (4,2)),  
    ADD (PROJ_NUM     CHAR(3));
```

9. Write the SQL code to enter an EMP_PCT value of 3.85 for the person whose employee number (EMP_NUM) is 103. Next, enter the remaining EMP_PCT values shown in Figure Q7.9:

FIGURE Q7.9 The Contents of the EMP_2 Table

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_HIREDATE	JOB_CODE	EMP_PCT	PROJ_NUM
101	News	John	G	08-Nov-00	502	5.00	
102	Senior	David	H	12-Jul-89	501	8.00	
103	Arbough	June	E	01-Dec-96	500	3.85	
104	Ramoras	Anne	K	15-Nov-87	501	10.00	
105	Johnson	Alice	K	01-Feb-93	502	5.00	
106	Smithfield	William		22-Jun-04	500	6.20	
107	Alonzo	Maria	D	10-Oct-93	500	5.15	
108	Washington	Ralph	B	22-Aug-91	501	10.00	
109	Smith	Larry	W	18-Jul-97	501	2.00	

```
UPDATE EMP_2
SET     EMP_PCT = 3.85
WHERE  EMP_NUM = '103';
```

To enter the remaining EMP_PCT values, use the following SQL statements:

```
UPDATE EMP_2
SET     EMP_PCT = 5.00
WHERE  EMP_NUM = '101';
```

```
UPDATE EMP_2
SET     EMP_PCT = 8.00
WHERE  EMP_NUM = '102';
```

Follow this format for the remaining rows.

10. Using a single command sequence, write the SQL code that will enter the project number (PROJ_NUM) = 18 for all employees whose job classification (JOB_CODE) is 500.

```
UPDATE EMP_2
SET     PROJ_NUM = '18'
WHERE  JOB_CODE = '500';
```

11. Using a single command sequence, write the SQL code that will enter the project number (PROJ_NUM) = 25 for all employees whose job classification (JOB_CODE) is 502 or higher. When you are done with questions 10 and 11, the EMP_2 table will contain the data shown in Figure Q7.11:

FIGURE Q7.11 The EMP_2 Table Contents After the Modifications

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_HIREDATE	JOB_CODE	EMP_PCT	PROJ_NUM
101	News	John	G	08-Nov-00	502	5.00	25
102	Senior	David	H	12-Jul-89	501	8.00	
103	Arbough	June	E	01-Dec-96	500	3.85	18
104	Ramoras	Anne	K	15-Nov-87	501	10.00	
105	Johnson	Alice	K	01-Feb-93	502	5.00	25
106	Smithfield	William		22-Jun-04	500	6.20	18
107	Alonzo	Maria	D	10-Oct-93	500	5.15	18
108	Washington	Ralph	B	22-Aug-91	501	10.00	
109	Smith	Larry	W	18-Jul-97	501	2.00	

(You may assume that the table has been saved again at this point!)

```
UPDATE EMP_2
SET     PROJ_NUM = '25'
WHERE  JOB_CODE >= '502'
```

12. Write the SQL code that will change the PROJ_NUM to 14 for those employees who were hired before January 1, 1994 and whose job code is at least 501. (You may assume that the table will be restored to its condition preceding this question.)

```
UPDATE EMP_2
SET     PROJ_NUM = '14'
WHERE  EMP_HIREDATE <= '01-Jan-94'
AND     JOB_CODE >= '501';
```

13. Write the two SQL command sequences required to:

There are many ways to accomplish both tasks. We are illustrating the shortest way to do the job next.

- a. Create a temporary table named TEMP_1 whose structure is composed of the EMP_2 attributes EMP_NUM and EMP_PCT.**

The SQL code shown in problem 13b contains the solution for problem 13a.

- b. Copy the matching EMP_2 values into the TEMP_1 table.**

```
CREATE TABLE TEMP_1 AS SELECT EMP_NUM, EMP_PCT FROM EMP_2;
```

An alternate way would be to create the table and then, use an INSERT with a sub-select to populate the rows.

```
CREATE TABLE TEMP_1 AS (  
  EMP_NUM  CHAR(3),  
  EMP_PCT  NUMBER(4,2));
```

```
INSERT INTO TEMP_1  
SELECT EMP_NUM, EMP_PCT FROM EMP_2;
```

14. Write the SQL command that will delete the newly created TEMP_1 table from the database.

```
DROP TABLE TEMP_1;
```

15. Write the SQL code required to list all employees whose last names start with *Smith*. In other words, the rows for both Smith and Smithfield should be included in the listing. Assume case sensitivity.

```
SELECT  *  
FROM    EMP_2  
WHERE   EMP_LNAME LIKE 'Smith%';
```

16. Using the EMPLOYEE, JOB, and PROJECT tables in the **Ch07_Review** database (see Figure Q7.1), write the SQL code that will produce the results shown in Figure Q7.16.

FIGURE Q7.16 The Query Results for Question 16

PROJ_NAME	PROJ_VALUE	PROJ_BALANCE	EMP_LNAME	EMP_FNAME	EMP_INITIAL	JOB_CODE	JOB_DESCRIPTION	JOB_CHG_HOUR
Rolling Tide	805000.00	500345.20	Senior	David	H	501	Systems Analyst	96.75
Evergreen	1453500.00	1002350.00	Arbough	June	E	500	Programmer	35.75
Starflight	2650500.00	2309880.00	Alonzo	Maria	D	500	Programmer	35.75
Amber Wave	3500500.00	2110346.00	Washington	Ralph	B	501	Systems Analyst	96.75

```
SELECT      PROJ_NAME, PROJ_VALUE, PROJ_BALANCE, EMPLOYEE.EMP_LNAME,
            EMP_FNAME, EMP_INITIAL, EMPLOYEE.JOB_CODE,
            JOB.JOB_DESCRIPTION,
            JOB.JOB_CHG_HOUR
FROM        PROJECT, EMPLOYEE, JOB
WHERE       EMPLOYEE.EMP_NUM = PROJECT.EMP_NUM
AND         JOB.JOB_CODE = EMPLOYEE.JOB_CODE;
```

17. Write the SQL code that will produce a virtual table named REP_1, containing the same information that was shown in Question 16.

```
CREATE VIEW REP_1 AS
SELECT      PROJ_NAME, PROJ_VALUE, PROJ_BALANCE, EMPLOYEE.EMP_LNAME,
            EMP_FNAME, EMP_INITIAL, EMPLOYEE.JOB_CODE,
            JOB.JOB_DESCRIPTION,
            JOB.JOB_CHG_HOUR
FROM        PROJECT, EMPLOYEE, JOB
WHERE       EMPLOYEE.EMP_NUM = PROJECT.EMP_NUM
AND         JOB.JOB_CODE = EMPLOYEE.JOB_CODE;
```

18. Write the SQL code to find the average bonus percentage in the EMP_2 table you created in question 8.

```
SELECT      AVG(EMP_PCT)
FROM        EMP_2;
```

The query output is shown in Figure Q7.18:

FIGURE Q7.18 Average Bonus Percentage

AvgOfEMP_PCT
6.13

19. Write the SQL code that will produce a listing for the data in the EMP_2 table in ascending order by the bonus percentage.

```
SELECT      *
FROM        EMP_2
ORDER BY    EMP_PCT;
```

20. Write the SQL code that will list only the different project numbers found in the EMP_2 table.

```
SELECT  DISTINCTC PROJ_NUM
FROM    EMP_2;
```

21. Write the SQL code to calculate the ASSIGN_CHARGE values in the ASSIGNMENT table in the [Ch07_Review](#) database. (See Figure Q7.1.) Note that ASSIGN_CHARGE is a derived attribute that is calculated by multiplying the ASSIGN_CHG_HR and the ASSIGN_HOURS.

```
UPDATE ASSIGNMENT
SET ASSIGN_CHARGE = ASSIGN_CHG_HR * ASSIGN_HOURS;
```

22. Using the data in the ASSIGNMENT table, write the SQL code that will, for each employee, yield the total number of hours worked and the total charges stemming from those hours worked. The results of running that query are shown in Figure Q7.22.

FIGURE Q7.22 Total Hours and Charges by Employee

EMP_NUM	EMP_LNAME	SumOfASSIGN_HOURS	SumOfASSIGN_CHARGE
101	News	3.1	387.50
103	Arbough	19.7	1664.65
104	Ramoras	11.9	1218.70
105	Johnson	12.5	1382.50
108	Washington	8.3	840.15
113	Joebrood	3.8	192.85
115	Bawangi	12.5	1276.75
117	Williamson	18.8	649.54

```
SELECT      ASSIGNMENT.EMP_NUM, EMPLOYEE.EMP_LNAME,
              Sum(ASSIGNMENT.ASSIGN_HOURS)          AS
SumOfASSIGN_HOURS,
              Sum(ASSIGNMENT.ASSIGN_CHARGE)         AS
SumOfASSIGN_CHARGE
FROM        EMPLOYEE, ASSIGNMENT
WHERE       EMPLOYEE.EMP_NUM = ASSIGNMENT.EMP_NUM
GROUP BY    ASSIGNMENT.EMP_NUM, EMPLOYEE.EMP_LNAME;
```

23. Write a query to produce the total number of hours and charges for each of the projects represented in the ASSIGNMENT table. The output is shown in Figure Q7.23.

FIGURE Q7.23 Total Hours and Charges by Project

PROJ_NUM	SumOfASSIGN_HOURS	SumOfASSIGN_CHARGE
15	20.5	1806.52
18	23.7	1544.80
22	27.0	2593.16
25	19.4	1668.16

```

SELECT      ASSIGNMENT.PROJ_NUM,
            Sum(ASSIGNMENT.ASSIGN_HOURS) AS SumOfASSIGN_HOURS,
            Sum(ASSIGNMENT.ASSIGN_CHARGE) AS SumOfASSIGN_CHARGE
FROM        ASSIGNMENT
GROUP BY    ASSIGNMENT.PROJ_NUM

```

24. Write the SQL code to generate the total hours worked and the total charges made by all employees. The results are shown in Figure Q7.24. (*Hint: This is a nested query. If you use Microsoft Access, you can generate the result by using the query output shown in Figure Q7.22 as the basis for the query that will produce the output shown in Figure Q7.24.*)

FIGURE Q7.24 Total Hours and Charges, All Employees

SumOfSumOfASSIGN_HOURS	SumOfSumOfASSIGN_CHARGE
90.6	7612.64

Solution A:

```

SELECT      Sum(SumOfASSIGN_HOURS) AS SumOfASSIGN_HOURS,
            Sum(SumOfASSIGN_CHARGE) AS SumOfASSIGN_CHARGE
FROM        Q23;

```

Solution B:

```

SELECT      Sum(SumOfASSIGN_HOURS) AS SumOfASSIGN_HOURS,
            Sum(SumOfASSIGN_CHARGE) AS SumOfASSIGN_CHARGE
FROM        Q22;

```

25. Write the SQL code to generate the total hours worked and the total charges made to all projects. The results should be the same as those shown in Figure Q7.24. (*Hint: This is a nested query. If you use Microsoft Access, you can generate the result by using the query output shown in Figure Q7.23 as the basis for this query.*)

```

SELECT      Sum(SumOfASSIGN_HOURS) AS SumOfASSIGN_HOURS,
            Sum(SumOfASSIGN_CHARGE) AS SumOfASSIGN_CHARGE
FROM        Q23;

```

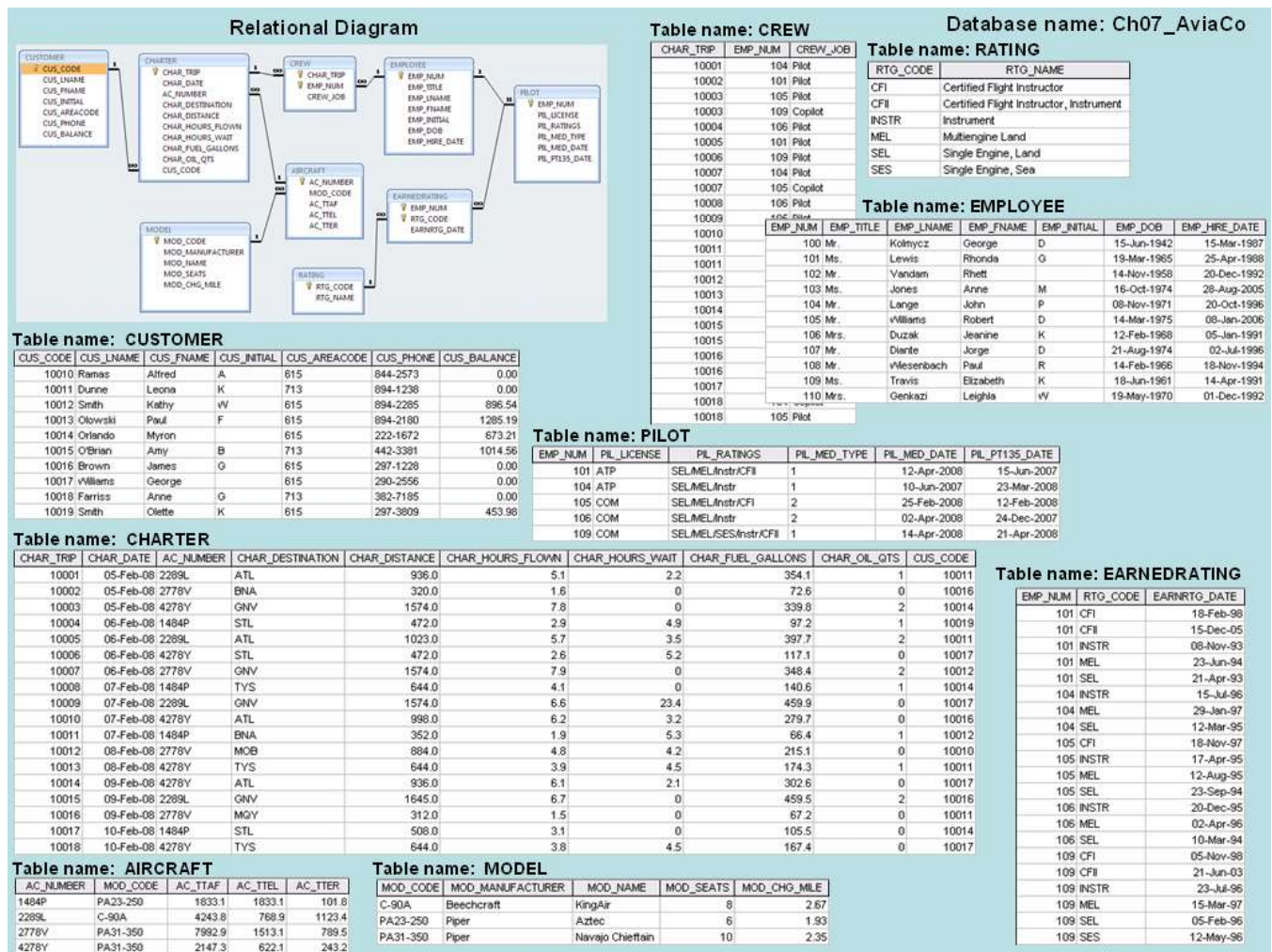
Problem Solutions



ONLINE CONTENT

Problems 1–15 are based on the [Ch07_AviaCo](#) database located in the Online Student Companion. This database is stored in Microsoft Access format. The Student Online Companion also includes SQL script files (Oracle and SQLServer) for all of the data sets used throughout the book.

Before you attempt to write any SQL queries, familiarize yourself with the [Ch07_AviaCo](#) database structure and contents shown in Figure P7.1. Although the relational schema does not show optionalities, keep in mind that all pilots are employees but not all employees are flight crew members. (Although in this database, the crew member assignments all involve pilots and copilots, the design is sufficiently flexible to accommodate crew member assignments—such as loadmasters and flight attendants—of people who are not pilots. That’s why the relationship between CHARTER and EMPLOYEE is implemented through CREW.) Note also that this design implementation does not include multivalued attributes. For example, multiple ratings such as Instrument and Certified Flight Instructor ratings are stored in the (composite) EARNEDRATINGS table. Nor does the CHARTER table include multiple crew assignments, which are properly stored in the CREW table.

FIGURE P7.1 Structure and Contents of the Ch07_AviaCo Database

1. Write the SQL code that will list the values for the first four attributes in the CHARTER table.

```
SELECT CHARTER.CHAR_TRIP, CHARTER.CHAR_DATE, CHARTER.AC_NUMBER,
       CHARTER.CHAR_DESTINATION
FROM   CHARTER;
```

2. Using the contents of the CHARTER table, write the SQL query that will produce the output shown in Figure P7.2. Note that the output is limited to selected attributes for aircraft number 2778V.

FIGURE P7.2 Problem 2 Query Results

CHAR_DATE	AC_NUMBER	CHAR_DESTINATION	CHAR_DISTANCE	CHAR_HOURS_FLOWN
05-Feb-08	2778V	BNA	320	1.6
06-Feb-08	2778V	GNV	1574	7.9
08-Feb-08	2778V	MOB	884	4.8
09-Feb-08	2778V	MQY	312	1.5

```

SELECT      DISTINCTROW CHARTER.CHAR_DATE, CHARTER.AC_NUMBER,
            CHARTER.CHAR_DESTINATION, CHARTER.CHAR_DISTANCE,
            CHARTER.CHAR_HOURS_FLOWN
FROM        CHARTER
WHERE       CHARTER.AC_NUMBER)="2778V";

```

3. Create a virtual table (named AC2778V) containing the output presented in Problem 2.

Note to Access users: Access does not support views. The SQL code in Problem 2 provides the code necessary to produce the view – but we have not created the view itself. You can add the CREATE VIEW component if you use Oracle or SQL Server.

Also, MS Access adds multiple parentheses around the condition when you run the query shown in Problem 2, rewriting the last line as

```
WHERE (((CHARTER.AC_NUMBER)='2778V'));
```

Finally, Access does not maintain the spacing we have shown here – you can write the SQL as we have shown, but Access will string the code together.



ONLINE CONTENT

You can find the queries in the teacher's version of the [Ch07_AviaCo](#) database.

4. Produce the output shown in Figure P7.4 for aircraft 2778V. Note that this output includes data from the CHARTER and CUSTOMER tables. (*Hint: Use a JOIN in this query.*)

FIGURE P7.4 Problem 4 Query Results

CHAR_DATE	AC_NUMBER	CHAR_DESTINATION	CUS_LNAME	CUS_AREACODE	CUS_PHONE
08-Feb-08	2778V	MOB	Ramas	615	844-2573
09-Feb-08	2778V	MQY	Dunne	713	894-1238
06-Feb-08	2778V	GNV	Smith	615	894-2285
05-Feb-08	2778V	BNA	Brown	615	297-1228

```
SELECT    DISTINCTROW CHARTER.CHAR_DATE, CHARTER.AC_NUMBER,
          CHARTER.CHAR_DESTINATION, CUSTOMER.CUS_LNAME,
          CUSTOMER.CUS_AREACODE, CUSTOMER.CUS_PHONE
FROM      CUSTOMER, CHARTER
WHERE     CUSTOMER.CUS_CODE = CHARTER.CUS_CODE
AND CHARTER.AC_NUMBER='2778V';
```

5. Produce the output shown in Figure P7.5. The output, derived from the CHARTER and MODEL tables, is limited to February 6, 2008. (*Hint: The JOIN passes through another table. Note that the “connection” between CHARTER and MODEL requires the existence of AIRCRAFT, because the CHARTER table does not contain a foreign key to MODEL. However, CHARTER does contain AC_NUMBER, a foreign key to AIRCRAFT, which contains a foreign key to MODEL.*)

FIGURE P7.5 Problem 5 Query Results

CHAR_DATE	CHAR_DESTINATION	AC_NUMBER	MOD_NAME	MOD_CHG_MILE
06-Feb-08	STL	1484P	Aztec	1.93
06-Feb-08	ATL	2289L	KingAir	2.67
06-Feb-08	STL	4278Y	Navajo Chieftain	2.35
06-Feb-08	GNV	2778V	Navajo Chieftain	2.35

```
SELECT    CHARTER.CHAR_DATE, CHARTER.CHAR_DESTINATION,
          CHARTER.AC_NUMBER, MODEL.MOD_NAME, MODEL.MOD_CHG_MILE
FROM      MODEL, AIRCRAFT, CHARTER
WHERE     AIRCRAFT.AC_NUMBER = CHARTER.AC_NUMBER
AND MODEL.MOD_CODE = AIRCRAFT.MOD_CODE
AND CHARTER.CHAR_DATE=#2/6/2008#));
```

(Note that Access uses the “#” delimiter for dates. If you use Oracle, use apostrophes as delimiters.)

6. Modify the query in Problem 5 to include data from the CUSTOMER table. This time the output is limited to charter records generated since February 9, 2008. (The query results are shown in Figure P7.6.)

FIGURE P7.6 Problem 6 Query Results

CHAR_DATE	CHAR_DESTINATION	AC_NUMBER	MOD_NAME	MOD_CHG_MILE	CUS_LNAME
09-Feb-08	ATL	4278Y	Navajo Chieftain	2.35	Williams
09-Feb-08	MQY	2778V	Navajo Chieftain	2.35	Dunne
09-Feb-08	GNV	2289L	KingAir	2.67	Brown
10-Feb-08	TYS	4278Y	Navajo Chieftain	2.35	Williams
10-Feb-08	STL	1484P	Aztec	1.93	Orlando

```

SELECT    CHARTER.CHAR_DATE, CHARTER.CHAR_DESTINATION,
          AIRCRAFT.AC_NUMBER, MODEL.MOD_NAME, MODEL.MOD_CHG_MILE,
          CUSTOMER.CUS_LNAME
FROM      CHARTER, MODEL, AIRCRAFT, CUSTOMER
WHERE     (((CHARTER.CHAR_DATE)>=#2/9/2008#) AND
          ((AIRCRAFT.AC_NUMBER)=[CHARTER].[AC_NUMBER]) AND
          ((CUSTOMER.CUS_CODE)=[CHARTER].[CUS_CODE]) AND
          ((MODEL.MOD_CODE)=[AIRCRAFT].[MOD_CODE]))
ORDER BY CHARTER.CHAR_DATE;

```

(We have copied and pasted the Access SQL code to show this solution. If you use Oracle, delete the extra parentheses and use the '09-Feb-08' date delimiter.)

7. Modify the query in Problem 6 to produce the output shown in Figure P7.7. The date limitation in Problem 6 applies to this problem, too. Note that this query includes data from the CREW and EMPLOYEE tables. (*Note:* You may wonder why the date restriction seems to generate more records than it did in Problem 7. Actually, the number of (CHARTER) records is the same, but several records are listed twice to reflect a crew of two: a pilot and a copilot. For example, the record for the 09-Feb-2008 flight to GNV, using aircraft 2289L, required a crew consisting of a pilot (Lange) and a copilot (Lewis).)

FIGURE P7.7 Problem 7 Query Results

CHAR_DATE	CHAR_DESTINATION	AC_NUMBER	MOD_CHG_MILE	CHAR_DISTANCE	EMP_NUM	CREW_JOB	EMP_LNAME
09-Feb-08	GNV	2289L	2.67	1,645	104	Pilot	Lange
09-Feb-08	GNV	2289L	2.67	1,645	101	Copilot	Lewis
09-Feb-08	MQY	2778V	2.35	312	109	Pilot	Travis
09-Feb-08	MQY	2778V	2.35	312	105	Copilot	Williams
09-Feb-08	ATL	4278Y	2.35	936	106	Pilot	Duzak
10-Feb-08	STL	1484P	1.93	508	101	Pilot	Lewis
10-Feb-08	TYS	4278Y	2.35	644	105	Pilot	Williams
10-Feb-08	TYS	4278Y	2.35	644	104	Copilot	Lange

```

SELECT    CHARTER.CHAR_DATE, CHARTER.CHAR_DESTINATION,
          AIRCRAFT.AC_NUMBER, MODEL.MOD_CHG_MILE,
          CHARTER.CHAR_DISTANCE,
          CREW.EMP_NUM, CREW.CREW_JOB, EMPLOYEE.EMP_LNAME
FROM      CHARTER, AIRCRAFT, MODEL, CREW, EMPLOYEE
WHERE     CHARTER.CHAR_TRIP = CREW.CHAR_TRIP
AND      EMPLOYEE.EMP_NUM = CREW.EMP_NUM
AND      AIRCRAFT.AC_NUMBER = CHARTER.AC_NUMBER
AND      MODEL.MOD_CODE = AIRCRAFT.MOD_CODE
AND      CHARTER.CHAR_DATE>=#2/9/2008#
ORDER BY  CHARTER.CHAR_DATE, AIRCRAFT.AC_NUMBER;

```

8. Modify the query in Problem 5 to include the computed (derived) attribute “fuel per hour.”
Hint: It is possible to use SQL to produce computed “attributes” that are not stored in any table. For example, the following SQL query is perfectly acceptable:

```

SELECT CHAR_DISTANCE, CHAR_FUEL_GALLONS/CHAR_DISTANCE
FROM   CHARTER;

```

(The above query produces the “gallons per mile flown” value.) Use a similar technique on joined tables to produce the “gallons per hour” output shown in Figure P7.8. (Note that 67.2 gallons/1.5 hours produces 44.8 gallons per hour.)

Query output such as the “gallons per hour” result shown in Figure P7.8 provide managers with very important information. In this case, why is the fuel burn for the Navajo Chieftain 4278Y flown on 9-Feb-08 so much higher than the fuel burn for that aircraft on 10-Feb-06? Such a query result may lead to additional queries to find out who flew the aircraft or what special circumstances might have existed.

Is the fuel burn difference due to poor fuel management by the pilot, does it reflect an engine fuel metering problem, or was there an error in the fuel recording? The ability to generate useful query output is an important management asset.

FIGURE P7.8 Problem 8 Query Results

CHAR_DATE	AC_NUMBER	MOD_NAME	CHAR_HOURS_FLOWN	CHAR_FUEL_GALLONS	Expr1
09-Feb-08	2778V	Navajo Chieftain	1.5	67.2	44.8
09-Feb-08	2289L	KingAir	6.7	459.5	68.5820895522388
09-Feb-08	4278Y	Navajo Chieftain	6.1	302.6	49.6065573770492
10-Feb-08	4278Y	Navajo Chieftain	3.8	167.4	44.0526315789474
10-Feb-08	1484P	Aztec	3.1	105.5	34.0322580645161

NOTE

The output format is determined by the RDBMS you use. In this example, the Access software defaulted to an output heading labeled “Expr1” to indicate the expression resulting from the division:

[CHARTER]/[CHAR_FUEL_GALLONS]/[CHARTER]/[CHAR_HOURS]

created by its expression builder. Oracle defaults to the full division label. You should learn to control the output format with the help of your RDBMS’s utility software.

The SQL code solution is shown next:

```
SELECT    CHARTER.CHAR_DATE, CHARTER.CHAR_DESTINATION,
          AIRCRAFT.AC_NUMBER, MODEL.MOD_CHG_MILE,
          CHARTER.CHAR_DISTANCE,
          CREW.EMP_NUM, CREW.CREW_JOB, EMPLOYEE.EMP_LNAME
FROM      CHARTER, AIRCRAFT, MODEL, CREW, EMPLOYEE
WHERE     CHARTER.CHAR_TRIP = CREW.CHAR_TRIP
AND EMPLOYEE.EMP_NUM = CREW.EMP_NUM
AND AIRCRAFT.AC_NUMBER = CHARTER.AC_NUMBER
AND MODEL.MOD_CODE = AIRCRAFT.MOD_CODE
AND CHARTER.CHAR_DATE>=#2/9/2008#
ORDER BY  CHARTER.CHAR_DATE, AIRCRAFT.AC_NUMBER;
```

If you had used the MS Access expression builder and used the MS Access properties box to redefine the output format, you might produce a neater-looking output such as that shown in Figure P7.8A.

FIGURE P7.8A MS Access Formatted Output

CHAR_DATE	AC_NUMBER	MOD_NAME	CHAR_HOURS_FLOWN	CHAR_FUEL_GALLONS	Gallons/Hour
09-Feb-08	2778V	Navajo Chieftain	1.5	67.2	44.80
09-Feb-08	2289L	King Air	6.7	459.5	68.58
09-Feb-08	4278Y	Navajo Chieftain	6.1	302.6	49.61
10-Feb-08	4278Y	Navajo Chieftain	3.8	167.4	44.05
10-Feb-08	1484P	Aztec	3.1	105.5	34.03

The MS Access Expression Builder (see Figure P7.8B) was used to create the Gallons/Hour field in Figure P7.8A and the MS Access Properties Box (see Figure P7.8C) was used to define the output format.

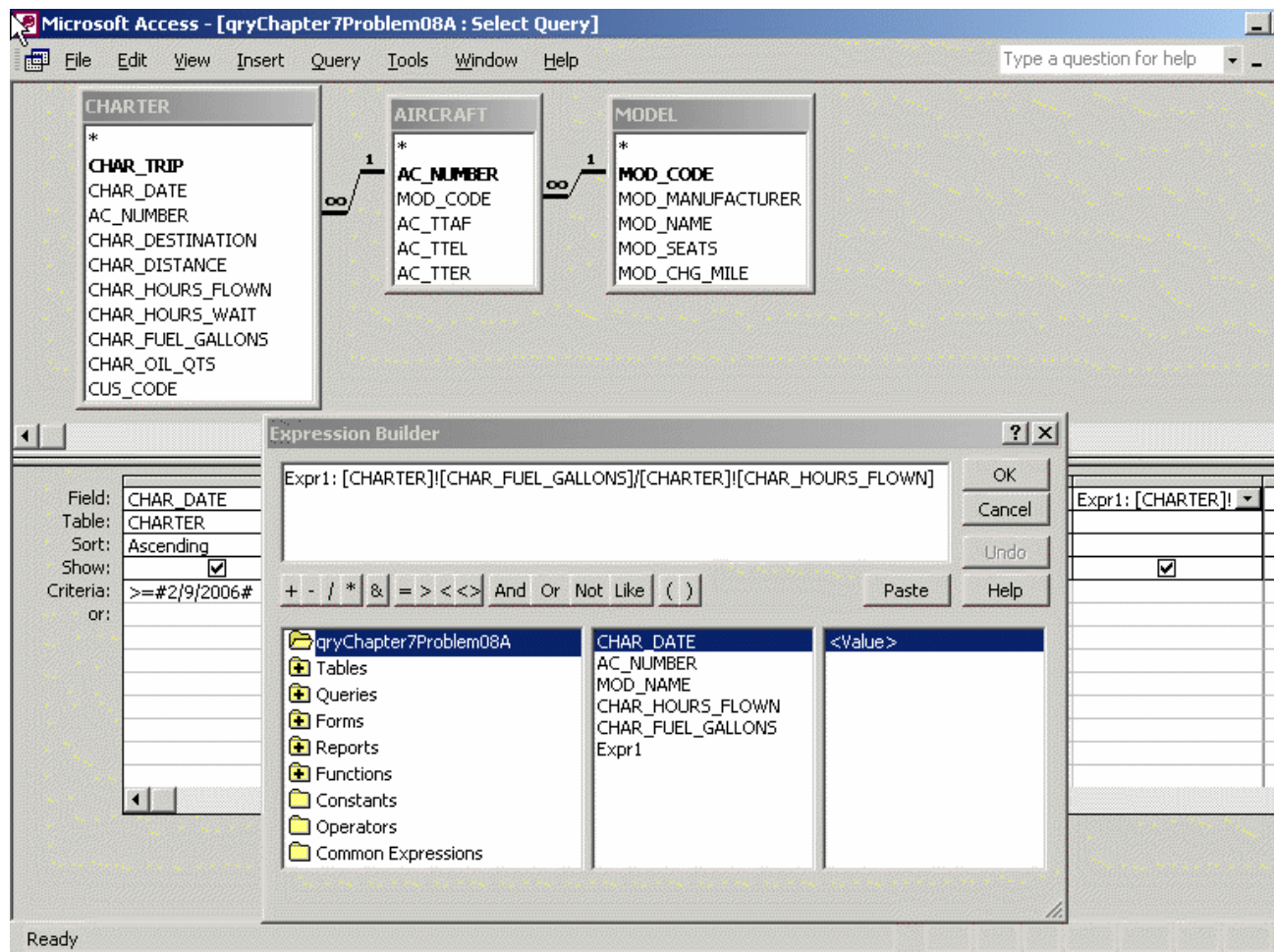
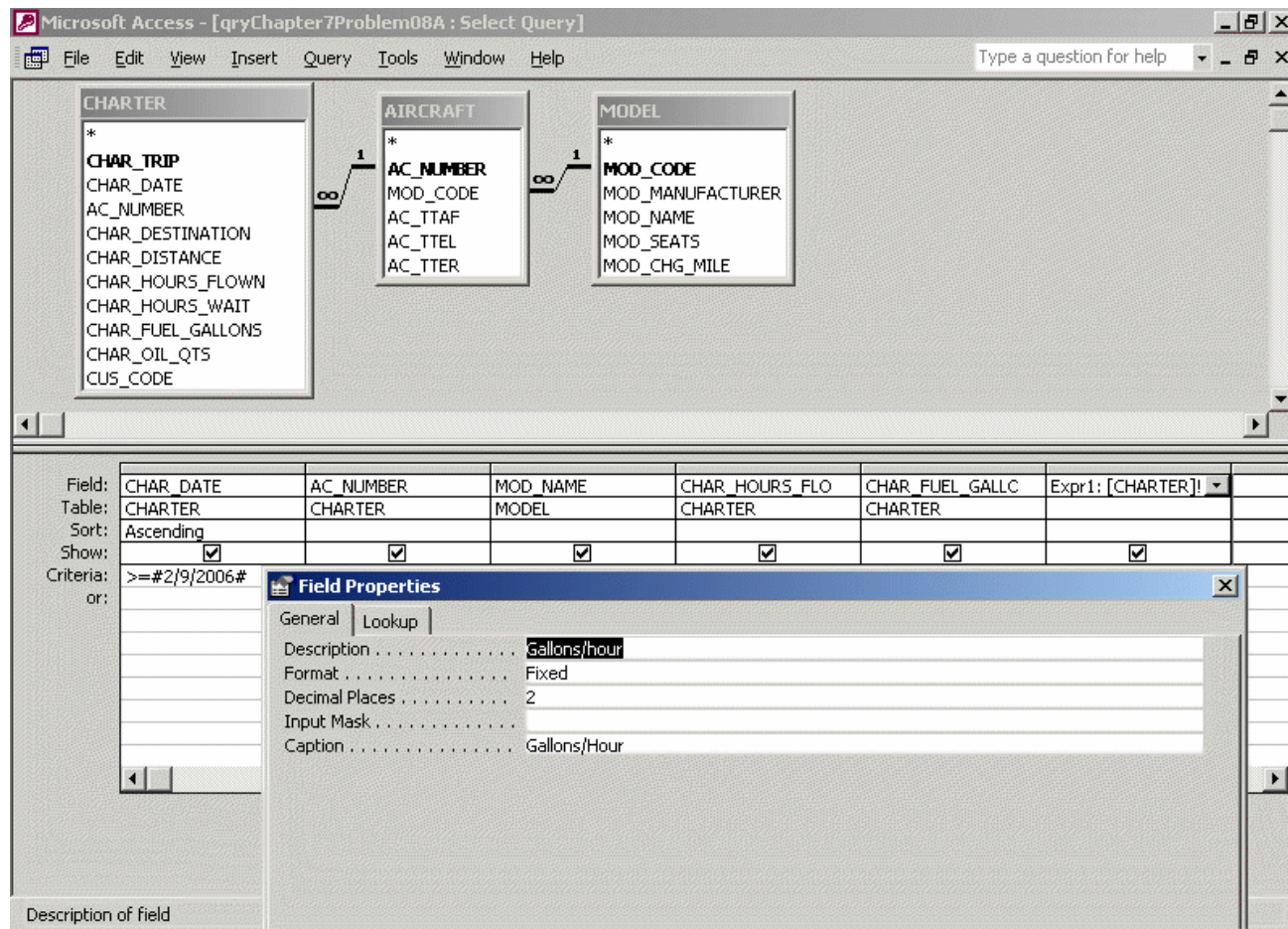
FIGURE P7.8B Using the Expression Builder

FIGURE P7.8C Using the Properties Box

9. Create a query to produce the output shown in Figure P7.9. Note that, in this case, the computed attribute requires data found in two different tables. (*Hint:* The MODEL table contains the charge per mile, and the CHARTER table contains the total miles flown.) Note also that the output is limited to charter records generated since February 9, 2006. In addition, the output is ordered by date and, within the date, by the customer's last name.

FIGURE P7.9 Problem 9 Query Results

CHAR_DATE	CUS_LNAME	CHAR_DISTANCE	MOD_CHG_MILE	Mileage Charge
09-Feb-08	Brown	1645	2.67	4392.15
09-Feb-08	Dunne	312	2.35	733.20
09-Feb-08	Williams	936	2.35	2199.60
10-Feb-08	Orlando	508	1.93	980.44
10-Feb-08	Williams	644	2.35	1513.40

The SQL query is shown on the next page.


```

SELECT    CHARTER.CHAR_DATE, CUSTOMER.CUS_LNAME,
          CHARTER.CHAR_DISTANCE, MODEL.MOD_CHG_MILE,
          CHARTER.CHAR_DISTANCE*MODEL.MOD_CHG_MILE AS Expr1
FROM      MODEL, CUSTOMER, AIRCRAFT, CHARTER
WHERE     AIRCRAFT.AC_NUMBER = CHARTER.AC_NUMBER
AND       CUSTOMER.CUS_CODE = CHARTER.CUS_CODE
AND       MODEL.MOD_CODE = AIRCRAFT.MOD_CODE
AND       CHARTER.CHAR_DATE>=#2/9/2008#
ORDER BY  CHARTER.CHAR_DATE, CUSTOMER.CUS_LNAME;

```

10. Use the techniques that produced the output in Problem 9 to produce the charges shown in Figure P7.10. The total charge to the customer is computed by:

- Miles flown * charge per mile
- Hours waited * \$50 per hour

The miles flown (CHAR_DISTANCE) value is found in the CHARTER table, the charge per mile (MOD_CHG_MILE) is found in the MODEL table, and the hours waited (CHAR_HOURS_WAIT) are found in the CHARTER table.

FIGURE P7.10 Problem 10 Query Results

CHAR_DATE	CUS_LNAME	Mileage Charge	Waiting Charge	Total Charge
09-Feb-08	Brown	4392.15	0.00	4392.15
09-Feb-08	Dunne	733.20	0.00	733.20
09-Feb-08	vWilliams	2199.60	105.00	2304.60
10-Feb-08	Orlando	980.44	0.00	980.44
10-Feb-08	vWilliams	1513.40	225.00	1738.40

```

SELECT    CHARTER.CHAR_DATE, CUSTOMER.CUS_LNAME,
          CHARTER.CHAR_DISTANCE*MODEL.MOD_CHG_MILE AS Expr1,
          CHARTER.CHAR_HOURS_WAIT*50 AS Expr2, Expr1+ Expr2 AS Expr3
FROM      MODEL, CUSTOMER, AIRCRAFT, CHARTER
WHERE     AIRCRAFT.AC_NUMBER = CHARTER.AC_NUMBER
AND       CUSTOMER.CUS_CODE = CHARTER.CUS_CODE
AND       MODEL.MOD_CODE = AIRCRAFT.MOD_CODE
AND       CHARTER.CHAR_DATE>=#2/9/2008#
ORDER BY  CHARTER.CHAR_DATE, CUSTOMER.CUS_LNAME;

```

11. Create the SQL query that will produce a list of customers who have an unpaid balance. The required output is shown in Figure P7.11. Note that the balances are listed in descending order.

FIGURE P7.11 A List of Customers with Unpaid Balances

CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_BALANCE
Olowski	Paul	F	1285.19
O'Brian	Amy	B	1014.56
Smith	Kathy	W	896.54
Orlando	Myron		673.21
Smith	Olette	K	453.98

```
SELECT    CUSTOMER.CUS_LNAME, CUSTOMER.CUS_FNAME,
          CUSTOMER.CUS_INITIAL, CUSTOMER.CUS_BALANCE
FROM      CUSTOMER
WHERE     CUSTOMER.CUS_BALANCE>0
ORDER BY  CUSTOMER.CUS_BALANCE DESC;
```

12. Find the average unpaid customer balance, the minimum balance, the maximum balance, and the total of the unpaid balances. The resulting values are shown in Figure P7.12.

FIGURE P7.12 Customer Balance Summary

Average Balance	Minimum Balance	Maximum Balance	Total Unpaid Bills
432.35	0.00	1285.19	4323.48

```
SELECT    Avg(CUSTOMER.CUS_BALANCE) AS AvgOfCUS_BALANCE,
          Min(CUSTOMER.CUS_BALANCE) AS MinOfCUS_BALANCE,
          Max(CUSTOMER.CUS_BALANCE) AS MaxOfCUS_BALANCE,
          Sum(CUSTOMER.CUS_BALANCE) AS SumOfCUS_BALANCE
FROM      CUSTOMER;
```

13. Using the CHARTER table as the source, group the aircraft data. Then use the SQL functions to produce the output shown in Figure P7.13. (Utility software was used to modify the headers, so your headers may look different.)

FIGURE P7.13 The AIRCRAFT Data Summary Statement

AC_NUMBER	Number of Trips	Total Distance	Average Distance	Total Hours	Average Hours
1484P	4	1976	494.0	12.0	3.0
2289L	4	5178	1294.5	24.1	6.0
2778V	4	3090	772.5	15.8	4.0
4278Y	6	5268	878.0	30.4	5.1

```

SELECT      CHARTER.AC_NUMBER, Count(CHARTER.AC_NUMBER) AS
            CountOfAC_NUMBER, Sum(CHARTER.CHAR_DISTANCE) AS
            SumOfCHAR_DISTANCE,
            Avg(CHARTER.CHAR_DISTANCE) AS AvgOfCHAR_DISTANCE,
            Sum(CHARTER.CHAR_HOURS_FLOWN) AS SumOfCHAR_HOURS_FLOWN,
            Avg(CHARTER.CHAR_HOURS_FLOWN) AS AvgOfCHAR_HOURS_FLOWN
FROM        CHARTER
GROUP BY    CHARTER.AC_NUMBER;

```

14. Write the SQL code to generate the output shown in Figure P7.14. Note that the listing includes all CHARTER flights that did not include a copilot crew assignment. (*Hint: The crew assignments are listed in the CREW table. Also note that the pilot's last name requires access to the EMPLOYEE table, while the MOD_CODE requires access to the MODEL table.*)

FIGURE P7.14 A Listing of All Charter Flights That Did Not Use a Copilot

CHAR_TRIP	CHAR_DATE	AC_NUMBER	MOD_NAME	CHAR_HOURS_FLOWN	EMP_LNAME	CREW_JOB
10001	05-Feb-08	2289L	KingAir	5.1	Lange	Pilot
10002	05-Feb-08	2778V	Navajo Chieftain	1.6	Lewis	Pilot
10004	06-Feb-08	1484P	Aztec	2.9	Duzak	Pilot
10005	06-Feb-08	2289L	KingAir	5.7	Lewis	Pilot
10006	06-Feb-08	4278Y	Navajo Chieftain	2.6	Travis	Pilot
10008	07-Feb-08	1484P	Aztec	4.1	Duzak	Pilot
10009	07-Feb-08	2289L	KingAir	6.6	Williams	Pilot
10010	07-Feb-08	4278Y	Navajo Chieftain	6.2	Wiesenbach	Pilot
10012	08-Feb-08	2778V	Navajo Chieftain	4.8	Lewis	Pilot
10013	08-Feb-08	4278Y	Navajo Chieftain	3.9	Williams	Pilot
10014	09-Feb-08	4278Y	Navajo Chieftain	6.1	Duzak	Pilot
10017	10-Feb-08	1484P	Aztec	3.1	Lewis	Pilot

The SQL code is shown next:

```

SELECT      CHARTER.CHAR_DATE, CHARTER.AC_NUMBER,
            MODEL.MOD_NAME,

```

```

    CHARTER.CHAR_HOURS_FLOWN, EMPLOYEE.EMP_LNAME,
    CREW.CREW_JOB
FROM      MODEL, AIRCRAFT, EMPLOYEE, CHARTER, CREW
WHERE     CHARTER.CHAR_TRIP = CREW.CHAR_TRIP
AND EMPLOYEE.EMP_NUM = CREW.EMP_NUM
AND AIRCRAFT.AC_NUMBER = CHARTER.AC_NUMBER
AND MODEL.MOD_CODE = AIRCRAFT.MOD_CODE
AND CREW.CHAR_TRIP Not In (SELECT DISTINCT CHAR_TRIP FROM CREW
WHERE     CREW_JOB = 'Copilot')
ORDER BY  CHARTER.CHAR_DATE;

```

15. Write a query that will list the ages of the employee and the date the query was run. The required output is shown in Figure P7.15. (As you can tell, the query was run on April 15, 2007, so the ages of the employee are current as of that date.)

FIGURE 7.15 Employee Ages and Date of Query

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_HIRE_DATE	EMP_DOB	Age	Query Date
100	Kolmycz	George	15-Mar-1987	15-Jun-1942	64	15-Apr-07
101	Lewis	Rhonda	25-Apr-1988	19-Mar-1965	42	15-Apr-07
102	Vandam	Rhett	20-Dec-1992	14-Nov-1958	48	15-Apr-07
103	Jones	Anne	28-Aug-2005	16-Oct-1974	32	15-Apr-07
104	Lange	John	20-Oct-1996	08-Nov-1971	35	15-Apr-07
105	Williams	Robert	08-Jan-2006	14-Mar-1975	32	15-Apr-07
106	Duzak	Jeanine	05-Jan-1991	12-Feb-1968	39	15-Apr-07
107	Diante	Jorge	02-Jul-1996	21-Aug-1974	32	15-Apr-07
108	Wiesenbach	Paul	18-Nov-1994	14-Feb-1966	41	15-Apr-07
109	Travis	Elizabeth	14-Apr-1991	18-Jun-1961	45	15-Apr-07
110	Genkazi	Leighla	01-Dec-1992	19-May-1970	36	15-Apr-07

```

SELECT      EMPLOYEE.EMP_NUM, EMPLOYEE.EMP_LNAME,
            EMPLOYEE.EMP_FNAME, EMPLOYEE.EMP_HIRE_DATE,
            EMPLOYEE.EMP_DOB, Int((Date()-EMPLOYEE.EMP_DOB)/365) AS
            Expr1, Date() AS Expr2
FROM        EMPLOYEE;

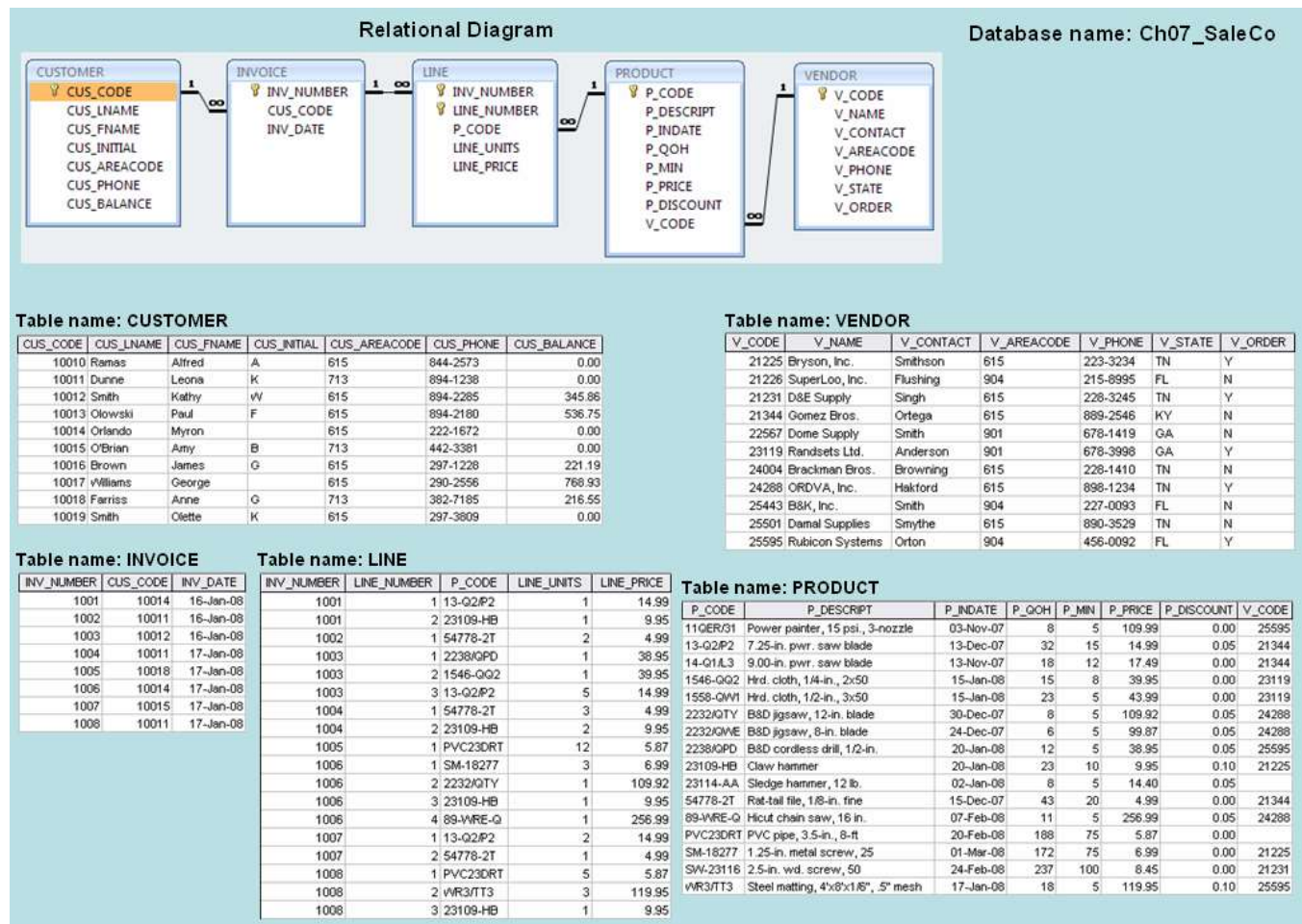
```



ONLINE CONTENT

Problems 16–33 are based on the [Ch07_SaleCo](#) database located in the Online Student Companion. This database is stored in Microsoft Access format. The Student Online Companion also includes SQL script files (Oracle and SQLServer) for all of the data sets used throughout the book.

The structure and contents of the [Ch07_SaleCo](#) database are shown in Figure P7.16. Use this database to answer the following problems. Save each query as QXX, where XX is the problem number.

FIGURE P7.16 The Ch07_SaleCo Database Structure and Contents

16. Write a query to count the number of invoices.

```
SELECT COUNT(*) FROM INVOICE;
```

17. Write a query to count the number of customers with a customer balance over \$500.

```
SELECT COUNT(*)
FROM CUSTOMER
WHERE CUS_BALANCE >500;
```

18. Generate a listing of all purchases made by the customers, using the output shown in Figure P7.18 as your guide. (*Hint: Use the ORDER BY clause to order the resulting rows as shown in Figure P7.18*)

FIGURE P7.18 List of Customer Purchases

CUS_CODE	INV_NUMBER	INV_DATE	P_DESCRIPT	LINE_UNITS	LINE_PRICE
10011	1002	16-Jan-08	Rat-tail file, 1/8-in. fine	2	4.99
10011	1004	17-Jan-08	Claw hammer	2	9.95
10011	1004	17-Jan-08	Rat-tail file, 1/8-in. fine	3	4.99
10011	1008	17-Jan-08	Claw hammer	1	9.95
10011	1008	17-Jan-08	PVC pipe, 3.5-in., 8-ft	5	5.87
10011	1008	17-Jan-08	Steel matting, 4'x8'x1/8", .5" mesh	3	119.95
10012	1003	16-Jan-08	7.25-in. pwr. saw blade	5	14.99
10012	1003	16-Jan-08	B&D cordless drill, 1/2-in.	1	38.95
10012	1003	16-Jan-08	Hrd. cloth, 1/4-in., 2x50	1	39.95
10014	1001	16-Jan-08	7.25-in. pwr. saw blade	1	14.99
10014	1001	16-Jan-08	Claw hammer	1	9.95
10014	1006	17-Jan-08	1.25-in. metal screw, 25	3	6.99
10014	1006	17-Jan-08	B&D jigsaw, 12-in. blade	1	109.92
10014	1006	17-Jan-08	Claw hammer	1	9.95
10014	1006	17-Jan-08	Hicut chain saw, 16 in.	1	256.99
10015	1007	17-Jan-08	7.25-in. pwr. saw blade	2	14.99
10015	1007	17-Jan-08	Rat-tail file, 1/8-in. fine	1	4.99
10018	1005	17-Jan-08	PVC pipe, 3.5-in., 8-ft	12	5.87

```

SELECT  INVOICE.CUS_CODE, INVOICE.INV_NUMBER, INVOICE.INV_DATE,
        PRODUCT.P_DESCRIPT, LINE.LINE_UNITS, LINE.LINE_PRICE
FROM    CUSTOMER, INVOICE, LINE, PRODUCT
WHERE   CUSTOMER.CUS_CODE = INVOICE.CUS_CODE
AND     INVOICE.INV_NUMBER = LINE.INV_NUMBER
AND     PRODUCT.P_CODE = LINE.P_CODE
ORDER BY INVOICE.CUS_CODE, INVOICE.INV_NUMBER, PRODUCT.P_DESCRIPT;
```

19. Using the output shown in Figure P7.19 as your guide, generate the listing of customer purchases, including the subtotals for each of the invoice line numbers. (*Hint: Modify the query format used to produce the listing of customer purchases in Problem 18, delete the INV_DATE column, and add the derived (computed) attribute LINE_UNITS * LINE_PRICE to calculate the subtotals.*)

FIGURE P7.19 Summary of Customer Purchases with Subtotals

CUS_CODE	INV_NUMBER	P_DESCRIPT	Units Bought	Unit Price	Subtotal
10011	1002	Rat-tail file, 1/8-in. fine	2	4.99	9.98
10011	1004	Claw hammer	2	9.95	19.90
10011	1004	Rat-tail file, 1/8-in. fine	3	4.99	14.97
10011	1008	Claw hammer	1	9.95	9.95
10011	1008	PVC pipe, 3.5-in., 8-ft	5	5.87	29.35
10011	1008	Steel matting, 4'x8'x1/8", .5" mesh	3	119.95	359.85
10012	1003	7.25-in. pwr. saw blade	5	14.99	74.95
10012	1003	B&D cordless drill, 1/2-in.	1	38.95	38.95
10012	1003	Hrd. cloth, 1/4-in., 2x50	1	39.95	39.95
10014	1001	7.25-in. pwr. saw blade	1	14.99	14.99
10014	1001	Claw hammer	1	9.95	9.95
10014	1006	1.25-in. metal screw, 25	3	6.99	20.97
10014	1006	B&D jigsaw, 12-in. blade	1	109.92	109.92
10014	1006	Claw hammer	1	9.95	9.95
10014	1006	Hicut chain saw, 16 in.	1	256.99	256.99
10015	1007	7.25-in. pwr. saw blade	2	14.99	29.98
10015	1007	Rat-tail file, 1/8-in. fine	1	4.99	4.99
10018	1005	PVC pipe, 3.5-in., 8-ft	12	5.87	70.44

```

SELECT      INVOICE.CUS_CODE, INVOICE.INV_NUMBER,
PRODUCT.P_DESCRIPT,
      LINE.LINE_UNITS AS [Units Bought], LINE.LINE_PRICE AS [Unit Price],
      LINE.LINE_UNITS*LINE.LINE_PRICE AS Subtotal
FROM        CUSTOMER, INVOICE, LINE, PRODUCT
WHERE       CUSTOMER.CUS_CODE = INVOICE.CUS_CODE
AND INVOICE.INV_NUMBER = LINE.INV_NUMBER
AND PRODUCT.P_CODE = LINE.P_CODE
ORDER BY   INVOICE.CUS_CODE, INVOICE.INV_NUMBER,
PRODUCT.P_DESCRIPT;

```


20. Modify the query used in Problem 19 to produce the summary shown in Figure P7.20.

FIGURE P7.20 Customer Purchase Summary

CUS_CODE	CUS_BALANCE	Total Purchases
10011	0.00	444.00
10012	345.86	153.85
10014	0.00	422.77
10015	0.00	34.97
10018	216.55	70.44

```
SELECT      INVOICE.CUS_CODE, CUSTOMER.CUS_BALANCE,
            Sum(LINE.LINE_UNITS*LINE.LINE_PRICE) AS [Total Purchases]
FROM        CUSTOMER, INVOICE, LINE
WHERE       INVOICE.INV_NUMBER = LINE.INV_NUMBER
AND CUSTOMER.CUS_CODE = INVOICE.CUS_CODE
GROUP BY    INVOICE.CUS_CODE, CUSTOMER.CUS_BALANCE;
```

21. Modify the query in Problem 20 to include the number of individual product purchases made by each customer. (In other words, if the customer's invoice is based on three products, one per LINE_NUMBER, you would count three product purchases. If you examine the original invoice data, you will note that customer 10011 generated three invoices, which contained a total of six lines, each representing a product purchase.) Your output values must match those shown in Figure P7.21.

FIGURE P7.21 Customer Total Purchase Amounts and Number of Purchases

CUS_CODE	CUS_BALANCE	Total Purchases	Number of Purchases
10011	0.00	444.00	6
10012	345.86	153.85	3
10014	0.00	422.77	6
10015	0.00	34.97	2
10018	216.55	70.44	1

```
SELECT      INVOICE.CUS_CODE, CUSTOMER.CUS_BALANCE,
            Sum(LINE.LINE_UNITS*LINE.LINE_PRICE) AS [Total Purchases],
            Count(*) AS [Number of Purchases]
FROM        CUSTOMER, INVOICE, LINE
WHERE       INVOICE.INV_NUMBER = LINE.INV_NUMBER
AND CUSTOMER.CUS_CODE = INVOICE.CUS_CODE
GROUP BY    INVOICE.CUS_CODE, CUSTOMER.CUS_BALANCE;
```

22. Use a query to compute the average purchase amount per product made by each customer. (*Hint: Use the results of Problem 21 as the basis for this query.*) Your output values must match those shown in Figure P7.22. Note that the Average Purchase Amount is equal to the Total Purchases divided by the Number of Purchases.

FIGURE P7.22 Average Purchase Amount by Customer

CUS_CODE	CUS_BALANCE	Total Purchases	Number of Purchases	Average Purchase Amount
10011	0.00	444.00	6	74.00
10012	345.86	153.85	3	51.28
10014	0.00	422.77	6	70.46
10015	0.00	34.97	2	17.48
10018	216.55	70.44	1	70.44

```

SELECT      INVOICE.CUS_CODE, CUSTOMER.CUS_BALANCE,
            Sum(LINE.LINE_UNITS*LINE.LINE_PRICE) AS [Total Purchases],
            Count(*) AS [Number of Purchases],
            AVG(LINE.LINE_UNITS*LINE.LINE_PRICE) AS [Average Purchase Amount]
FROM        CUSTOMER, INVOICE, LINE
WHERE       INVOICE.INV_NUMBER = LINE.INV_NUMBER
AND         CUSTOMER.CUS_CODE = INVOICE.CUS_CODE
GROUP BY    INVOICE.CUS_CODE, CUSTOMER.CUS_BALANCE;

```

23. Create a query to produce the total purchase per invoice, generating the results shown in Figure P7.23. The Invoice Total is the sum of the product purchases in the LINE that corresponds to the INVOICE.

FIGURE P7.23 Invoice Totals

INV_NUMBER	Invoice Total
1001	24.94
1002	9.98
1003	153.85
1004	34.87
1005	70.44
1006	397.83
1007	34.97
1008	399.15

```

SELECT      LINE.INV_NUMBER,
            Sum(LINE.LINE_UNITS*LINE.LINE_PRICE) AS [Invoice Total]
FROM        LINE
GROUP BY    LINE.INV_NUMBER;

```

24. Use a query to show the invoices and invoice totals as shown in Figure P7.24. (*Hint: Group by the CUS_CODE.*)

FIGURE P7.24 Invoice Totals by Customer

CUS_CODE	INV_NUMBER	Invoice Total
10011	1002	9.98
10011	1004	34.87
10011	1008	399.15
10012	1003	153.85
10014	1001	24.94
10014	1006	397.83
10015	1007	34.97
10018	1005	70.44

```

SELECT          CUS_CODE, LINE.INV_NUMBER AS INV_NUMVER,
                Sum(LINE.LINE_UNITS*LINE.LINE_PRICE)  AS   [Invoice
Total]
FROM            INVOICE, LINE
WHERE           INVOICE.INV_NUMBER = LINE.INV_NUMBER
GROUP BY       CUS_CODE, LINE.INV_NUMBER;
```

25. Write a query to produce the number of invoices and the total purchase amounts by customer, using the output shown in Figure P7.25 as your guide. (Compare this summary to the results shown in Problem 24.)

FIGURE P7.25 Number of Invoices and Total Purchase Amounts by Customer

CUS_CODE	Number of Invoices	Total Customer Purchases
10011	3	444.00
10012	1	153.85
10014	2	422.77
10015	1	34.97
10018	1	70.44

Note that a query may be used as the data source for another query. The following code is shown in [qryP7.25A](#) in your [Ch07_Saleco](#) database. Note that the data source is [qryP6-24](#).

```

SELECT          CUS_CODE,
                Count(INV_NUMBER) AS [Number of Invoices],
                AVG([Invoice Total]) AS [Average Invoice Amount],
                MAX([Invoice Total]) AS [Max Invoice Amount],
                MIN([Invoice Total]) AS [Min Invoice Amount],
                Sum([Invoice Total]) AS [Total Customer Purchases]
FROM            [qryP7-24]
```

```
GROUP BY [qryP7-24].CUS_CODE;
```

Instead of using another query as your data source, you can also use an alias. The following code is shown in **Oracle format**. You can also find the MS Access “alias” version in qryP7.25B in your **Ch07_SaleCo** database.)

```
SELECT      CUS_CODE,
COUNT(LINE.INV_NUMBER) AS [Number of Invoices],
AVG([Invoice Total]) AS [Average Invoice Amount],
MAX([Invoice Total]) AS [Max Invoice Amount],
MIN([Invoice Total]) AS [Min Invoice Amount],
Sum([Invoice Total]) AS [Total Customer Purchases]
FROM        (SELECT      CUS_CODE, LINE.INV_NUMBER AS INV_NUMBER,
Sum(LINE.LINE_UNITS*LINE.LINE_PRICE) AS [Invoice Total]
FROM        INVOICE, LINE
WHERE       INVOICE.INV_NUMBER = LINE.INV_NUMBER
GROUP BY    CUS_CODE, LINE.INV_NUMBER)
GROUP BY    CUS_CODE;
```

26. Using the query results in Problem 25 as your basis, write a query to generate the total number of invoices, the invoice total for all of the invoices, the smallest invoice amount, the largest invoice amount, and the average of all of the invoices. (*Hint: Check the figure output in Problem 25.*) Your output must match Figure P7.26.

FIGURE P7.26 Number of Invoices, Invoice Totals, Minimum,

**M
a
x
i
m
u
m
,
a
n
d
A
v
e**

Total Invoices	Total Sales	Minimum Sale	Largest Sale	Average Sale
8	1126.03	34.97	444.00	225.21

```

SELECT      Count([qryP7-24].[INV_NUMBER]) AS [Total Invoices],
            Sum([qryP7-24].[Invoice Total]) AS [Total Sales],
            Min([qryP7-24].[Invoice Total]) AS [Minimum Sale],
            Max([qryP7-24].[Invoice Total]) AS [Largest Sale],
            Avg([qryP7-24].[Invoice Total]) AS [Average Sale]
FROM        [qryP7-24];
    
```

27. List the balance characteristics of the customers who have made purchases during the current invoice cycle—that is, for the customers who appear in the INVOICE table. The results of this query are shown in Figure P7.27.

FIGURE P7.27 Balances for Customers who Made Purchases

CUS_CODE	CUS_BALANCE
10011	0.00
10012	345.86
10014	0.00
10015	0.00
10018	216.55

```
SELECT    CUS_CODE, CUS_BALANCE
FROM      CUSTOMER
WHERE     CUSTOMER.CUS_CODE IN
          (SELECT DISTINCT CUS_CODE FROM INVOICE );
```

28. Using the results of the query created in Problem 27, provide a summary of customer balance characteristics as shown in Figure P7.28.

FIGURE P7.28 Balance Summary for Customers Who Made Purchases

Minimum Balance	Maximum Balance	Average Balance
0	345.86	112.48

```
SELECT    MIN(CUS_BALANCE) AS [Minimum Balance],
          MAX(CUS_BALANCE) AS [Maximum Balance],
          AVG(CUS_BALANCE) AS [Average Balance]
FROM      (SELECT CUS_CODE, CUS_BALANCE
          FROM    CUSTOMER
          WHERE   CUSTOMER.CUS_CODE IN (SELECT DISTINCT
CUS_CODE
FROM INVOICE) );
```

29. Create a query to find the customer balance characteristics for all customers, including the total of the outstanding balances. The results of this query are shown in Figure P7.29.

FIGURE P7.29 Customer Balance Summary for All Customers

Total Balances	Minimum Balance	Maximum Balance	Average Balance
2089.28	0.00	768.93	208.93

```
SELECT    Sum(CUS_BALANCE) AS [Total Balance], Min(CUS_BALANCE) AS
          [Minimum Balance], Max(CUS_BALANCE) AS [Maximum Balance],
          Avg(CUS_BALANCE) AS [Average Balance]
FROM      CUSTOMER;
```

30. Find the listing of customers who did not make purchases during the invoicing period. Your output must match the output shown in Figure P7.30.

FIGURE P7.30 Customer Balances for Customers Who Did Not Make Purchases

CUS_CODE	CUS_BALANCE
10010	0.00
10013	536.75
10016	221.19
10017	768.93
10019	0.00

```
SELECT    CUS_CODE, CUS_BALANCE
FROM      CUSTOMER
WHERE     CUSTOMER.CUS_CODE NOT IN
          (SELECT DISTINCT CUS_CODE FROM INVOICE);
```

31. Find the customer balance summary for all customers who have not made purchases during the current invoicing period. The results are shown in Figure P7.31.

FIGURE P7.31 Summary of Customer Balances for

**Custom
ers
Who
Did Not
Make
Purchas
es**

Total Balance	Minimum Balance	Maximum Balance	Average Balance
1526.87	0.00	768.93	305.37

```
SELECT    SUM(CUS_BALANCE) AS [Total Balance],
          MIN(CUS_BALANCE) AS [Minimum Balance],
          MAX(CUS_BALANCE) AS [Maximum Balance],
          AVG(CUS_BALANCE) AS [Average Balance]
FROM      (SELECT CUS_CODE, CUS_BALANCE
FROM      CUSTOMER
```


WHERE

```
    CUST  
    OMER.CU  
    S_CODE  
    NOT IN  
    (SELECT  
    DISTINCT  
    CUS_COD  
    E FROM  
    INVOICE)  
    );
```

32. Create a query to produce the summary of the value of products currently in inventory. Note that the value of each product is produced by the multiplication of the units currently in inventory and the unit price. Use the ORDER BY clause to match the order shown in Figure P7.32.

FIGURE P7.32 Value of Products in Inventory

P_DESCRIPTION	P_QOH	P_PRICE	Subtotal
Power painter, 15 psi., 3-nozzle	8	109.99	879.92
7.25-in. pwr. saw blade	32	14.99	479.68
9.00-in. pwr. saw blade	18	17.49	314.82
Hrd. cloth, 1/4-in., 2x50	15	39.95	599.25
Hrd. cloth, 1/2-in., 3x50	23	43.99	1011.77
B&D jigsaw, 12-in. blade	8	109.92	879.36
B&D jigsaw, 8-in. blade	6	99.87	599.22
B&D cordless drill, 1/2-in.	12	38.95	467.40
Claw hammer	23	9.95	228.85
Sledge hammer, 12 lb.	8	14.40	115.20
Rat-tail file, 1/8-in. fine	43	4.99	214.57
Hicut chain saw, 16 in.	11	256.99	2826.89
PVC pipe, 3.5-in., 8-ft	188	5.87	1103.56
1.25-in. metal screw, 25	172	6.99	1202.28
2.5-in. wd. screw, 50	237	8.45	2002.65
Steel matting, 4'x8'x1/8", .5" mesh	18	119.95	2159.10

```
SELECT    P_DESCRIPTION, P_QOH, P_PRICE, P_QOH*P_PRICE AS Subtotal
FROM      PRODUCT;
```

33. Using the results of the query created in Problem 32, find the total value of the product inventory. The results are shown in Figure P7.33.

FIGURE P7.33 Total Value of All Products in Inventory

Total Value of Inventory
5084.52

```
SELECT    SUM(P_QOH*P_PRICE) AS [Total Value of Inventory]
FROM      PRODUCT;
```